

مستند الگوریتم‌ها AI Chat SaaS

Algorithms Documentation

تشریح الگوریتم‌های کلیدی سامانه چت پشتیبانی چندمستاجری، موتور دانش هوش مصنوعی، خزنده سایت، کنترل دسترسی، محدودیت پلن، نرخ درخواست و اعتبارسنجی فایل

AI Chat SaaS	نام پروژه:
مستند الگوریتم‌ها	نوع سند:
1.0	نسخه سند:
۱۴ جولای ۲۰۲۶	تاریخ:
مبین یوسفی	نویسنده/تهیه‌کننده:
doc/LaTeX/06-Algorithms/06-Algorithms.tex	مسیر سند:

این سند مطابق بخش «مستند الگوریتم‌ها» در Rahnama.md و بر پایه کدهای واقعی پروژه، ArchitectureForMobin.md و ReportForSale.md تهیه شده است.

فهرست مطالب

۲	۱ هدف و دامنه سند
۲	۲ خلاصه الگوریتم‌های کلیدی
۴	۳ الگوریتم ۱: احراز هویت JWT و ابطال توکن
۴	۱.۳ هدف
۴	۲.۳ ورودی
۴	۳.۳ خروجی
۴	۴.۳ شبه‌کد
۵	۵.۳ فلوچارت
۵	۶.۳ پیچیدگی
۵	۷.۳ مزایا
۵	۸.۳ محدودیت‌ها
۶	۴ الگوریتم ۲: کنترل دسترسی چندمستاجر به سایت
۶	۱.۴ هدف
۶	۲.۴ ورودی
۶	۳.۴ خروجی
۶	۴.۴ شبه‌کد
۶	۵.۴ فلوچارت
۷	۶.۴ پیچیدگی
۷	۷.۴ مزایا
۷	۸.۴ محدودیت‌ها
۸	۵ الگوریتم ۳: شروع یا بازیابی مکالمه در Widget
۸	۱.۵ هدف
۸	۲.۵ ورودی
۸	۳.۵ خروجی
۸	۴.۵ شبه‌کد
۹	۵.۵ فلوچارت
۹	۶.۵ پیچیدگی
۹	۷.۵ مزایا
۹	۸.۵ محدودیت‌ها

۱۰	۶ الگوریتم ۴: اعمال محدودیت‌های پلن
۱۰	۱.۶ هدف
۱۰	۲.۶ ورودی
۱۰	۳.۶ خروجی
۱۰	۴.۶ شبه‌کد
۱۱	۵.۶ فلوچارت
۱۱	۶.۶ پیچیدگی
۱۱	۷.۶ مزایا
۱۱	۸.۶ محدودیت‌ها
۱۲	۷ الگوریتم ۵: محدودسازی نرخ درخواست
۱۲	۱.۷ هدف
۱۲	۲.۷ ورودی
۱۲	۳.۷ خروجی
۱۲	۴.۷ شبه‌کد
۱۳	۵.۷ فلوچارت
۱۳	۶.۷ پیچیدگی
۱۳	۷.۷ مزایا
۱۳	۸.۷ محدودیت‌ها
۱۴	۸ الگوریتم ۶: خزیدن سایت و ساخت دانش
۱۴	۱.۸ هدف
۱۴	۲.۸ ورودی
۱۴	۳.۸ خروجی
۱۴	۴.۸ شبه‌کد
۱۵	۵.۸ فلوچارت
۱۵	۶.۸ پیچیدگی
۱۶	۷.۸ مزایا
۱۶	۸.۸ محدودیت‌ها
۱۷	۹ الگوریتم ۷: نرمال‌سازی، قطعه‌بندی و استخراج واژه‌های مهم
۱۷	۱.۹ هدف
۱۷	۲.۹ ورودی
۱۷	۳.۹ خروجی

۱۷	شبکه‌کد	۴.۹
۱۸	فلوچارت	۵.۹
۱۸	پیچیدگی	۶.۹
۱۸	مزایا	۷.۹
۱۸	محدودیت‌ها	۸.۹

۱۰ الگوریتم ۸: بازیابی و رتبه‌بندی پاسخ AI

۱۹	هدف	۱.۱۰
۱۹	ورودی	۲.۱۰
۱۹	خروجی	۳.۱۰
۱۹	شبکه‌کد	۴.۱۰
۲۰	فلوچارت	۵.۱۰
۲۱	پیچیدگی	۶.۱۰
۲۱	مزایا	۷.۱۰
۲۱	محدودیت‌ها	۸.۱۰

۱۱ الگوریتم ۹: پاسخ خودکار AI در Widget

۲۲	هدف	۱.۱۱
۲۲	ورودی	۲.۱۱
۲۲	خروجی	۳.۱۱
۲۲	شبکه‌کد	۴.۱۱
۲۳	فلوچارت	۵.۱۱
۲۴	پیچیدگی	۶.۱۱
۲۴	مزایا	۷.۱۱
۲۴	محدودیت‌ها	۸.۱۱

۱۲ الگوریتم ۱۰: تولید پیشنهاد پاسخ برای Agent

۲۵	هدف	۱.۱۲
۲۵	ورودی	۲.۱۲
۲۵	خروجی	۳.۱۲
۲۵	شبکه‌کد	۴.۱۲
۲۶	فلوچارت	۵.۱۲
۲۶	پیچیدگی	۶.۱۲
۲۶	مزایا	۷.۱۲

۲۷ ۸.۱۲ محدودیت‌ها

۱۳ الگوریتم ۱۱: اعتبارسنجی و ذخیره امن فایل پیوست

۲۸ ۱.۱۳ هدف

۲۸ ۲.۱۳ ورودی

۲۸ ۳.۱۳ خروجی

۲۸ ۴.۱۳ شبه‌کد

۲۹ ۵.۱۳ فلوچارت

۲۹ ۶.۱۳ پیچیدگی

۲۹ ۷.۱۳ مزایا

۳۰ ۸.۱۳ محدودیت‌ها

۱۴ الگوریتم ۱۲: چرخه سوال بی‌پاسخ و بهبود دانش

۳۱ ۱.۱۴ هدف

۳۱ ۲.۱۴ ورودی

۳۱ ۳.۱۴ خروجی

۳۱ ۴.۱۴ شبه‌کد

۳۲ ۵.۱۴ فلوچارت

۳۲ ۶.۱۴ پیچیدگی

۳۲ ۷.۱۴ مزایا

۳۲ ۸.۱۴ محدودیت‌ها

۱۵ جمع‌بندی فنی

۳۲

۱ هدف و دامنه سند

این سند الگوریتم‌های اصلی پروژه AI Chat SaaS را به صورت فنی، قابل بررسی و قابل ارائه مستند می‌کند. مطابق راهنمای Rahnama.md، برای هر الگوریتم موارد زیر ارائه شده است:

- هدف الگوریتم
- ورودی‌ها
- خروجی‌ها
- شبه‌کد
- فلوچارت متنی
- پیچیدگی زمانی و فضایی
- مزایا
- محدودیت‌ها

تمرکز سند روی الگوریتم‌هایی است که هسته فنی محصول را می‌سازند: احراز هویت، JWT کنترل دسترسی چندمستاجر، ایجاد مکالمه، widget محدودیت پلن، rate limiting، خزیدن محتوای سایت، آماده‌سازی دانش، جست‌وجو و رتبه‌بندی پاسخ، AI پاسخ خودکار، AI پیشنهاد پاسخ برای agent و اعتبارسنجی امن فایل.

۲ خلاصه الگوریتم‌های کلیدی

الگوریتم	فایل‌های اصلی	نقش در محصول
احراز هویت JWT و revoke	auth.php, jwt.php	حفاظت از API پل و کنترل نشست کاربران.
کنترل دسترسی سایت	site-access.php	جداسازی tenant/site و جلوگیری از دسترسی غیرمجاز. ها. agent
شروع مکالمه widget	conversation-start.php	ایجاد یا بازیابی مکالمه فعال. visitor.
محدودیت پلن	plan-limits.php	اعمال سقف سایت، agent، مکالمه و قابلیت‌های AI.
limiting Rate	rate-limit.php	جلوگیری از brute force و spam در login/widget/AI.
خزنده دانش سایت	ai-crawl-start.php, ai-crawler.php	ساخت knowledge base از صفحات سایت مشتری.

پردازش متن و term extraction	ai-crawler.php ai-answer-engine.php	آماده‌سازی متن، توکن، intent و term برای جست‌وجو.
بازیابی و رتبه‌بندی پاسخ AI	ai-answer-engine.php	پیدا کردن بهترین پاسخ از دانش دستی و crawled content.
پاسخ خودکار AI	ai-reply.php	پاسخ مستقیم در wid- get هنگام آفلاین بودن پشتیبانی.
پیشنهاد پاسخ agent	ai-suggestion-generate.php	ساخت draft برای hu- agent man با confi- source. و dence
اعتبارسنجی upload	upload.php	جلوگیری از فایل خطرناک و ذخیره امن attach- ment.

۳ الگوریتم ۱: احراز هویت JWT و ابطال توکن

۱.۳ هدف

هدف این الگوریتم تولید و اعتبارسنجی توکن JWT با امضای HS256 و کنترل ابطال نشست‌ها با token_version است. این الگوریتم باعث می‌شود های API پزل بدون session سمت سرور کار کنند، اما همچنان تغییر رمز یا logout-all بتواند توکن‌های قبلی را از اعتبار خارج کند.

۲.۳ ورودی

- در زمان تولید: شناسه کاربر، tenant ایمیل، نقش، token_version و زمان انقضا.
- در زمان اعتبارسنجی: مقدار header از نوع Authorization: Bearer <token>.
- تنظیمات: JWT_SECRET، audience issuer و حداکثر TTL.

۳.۳ خروجی

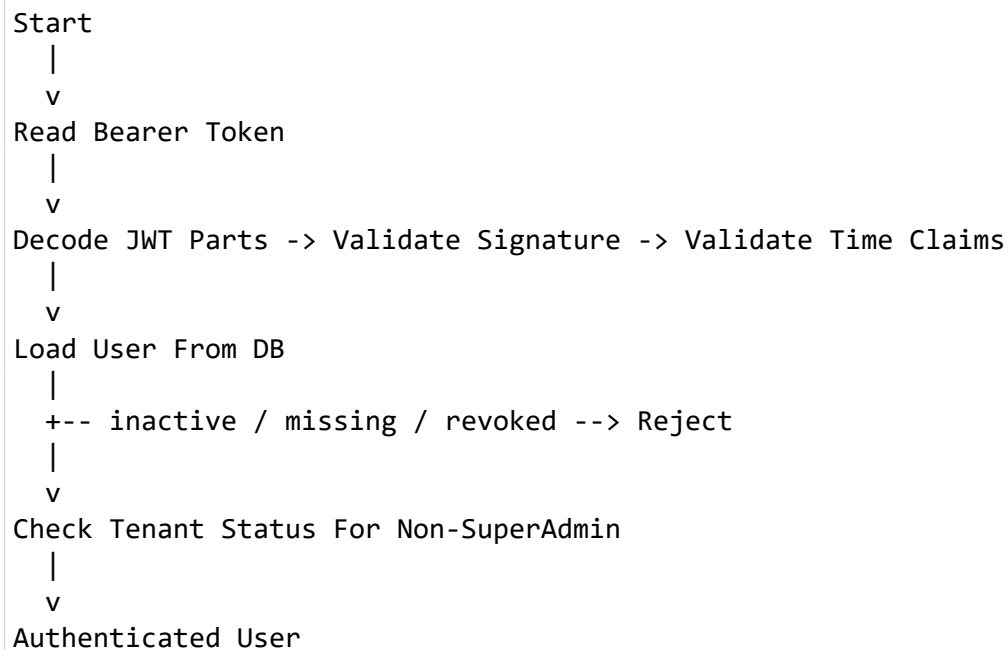
- در تولید: رشته JWT امضا شده.
- در اعتبارسنجی موفق: payload و سپس اطلاعات کاربر فعال.
- در خطا: پاسخ 401، 403 یا 500 بسته به نوع خطا.

۴.۳ شبه‌کد

```
Algorithm JWT_AUTHENTICATION
Input: Authorization header, database connection
Output: authenticated user or error

1. Extract Bearer token from Authorization header
2. Split token into header, payload, signature
3. Base64URL-decode header and payload
4. Verify header.typ == JWT and header.alg == HS256
5. Recompute HMAC-SHA256(header.payload, JWT_SECRET)
6. Compare expected signature with token signature using constant-time compare
7. Validate exp, iat, nbf, jti, max TTL, issuer and audience
8. Load user by payload.sub from database
9. Reject if user is inactive
10. Compare payload.token_version with users.token_version
11. Compare token email/role with current database values
12. If user is not super_admin, require active tenant
13. Return normalized user object
```

۵.۳ فلوجارت



۶.۳ پیچیدگی

پیچیدگی زمانی این الگوریتم $O(1)$ نسبت به اندازه دیتابیس است، چون فقط یک کاربر با کلید اصلی خوانده می‌شود. پیچیدگی فضایی نیز $O(1)$ است. هزینه رمزنگاری HMAC نسبت به طول توکن خطی است، اما طول توکن محدود به ۴۰۹۶ کاراکتر است.

۷.۳ مزایا

- stateless بودن احراز هویت.
- امکان revoke عملی با token_version.
- کنترل، issuer، audience، TTL و skew. clock.
- استفاده از hash_equals برای مقایسه امن امضا.

۸.۳ محدودیت‌ها

- امنیت کاملاً به محرمانه و قوی بودن JWT_SECRET وابسته است.
- blacklist مجزا برای tti وجود ندارد؛ ابطال در سطح کاربر با token_version انجام می‌شود.
- token refresh مستقل در کد فعلی دیده نمی‌شود.

۴ الگوریتم ۲: کنترل دسترسی چندمستاجر به سایت

۱.۴ هدف

هدف الگوریتم، تعیین این است که کاربر لاگین‌شده اجازه دسترسی به یک `site_id` را دارد یا نه. این الگوریتم هسته جداسازی `tenant`ها و جلوگیری از IDOR در های `agent endpoint` و `customer` است.

۲.۴ ورودی

- کاربر احرازشده شامل `id`، `role` و `tenant_id`.
- شناسه سایت `site_id`.
- جداول `agent_site_access` و `sites`.

۳.۴ خروجی

- مقدار `boolean` در `user_can_access_site`.
- یا پاسخ 403 در `require_site_access`.

۴.۴ شبه‌کد

```
Algorithm SITE_ACCESS
Input: user, site_id
Output: allowed or denied

1. If user.role == super_admin:
    return allowed
2. If user.role == customer_admin:
    Find site where site.id = site_id and site.tenant_id = user.tenant_id
    If found return allowed else denied
3. If user.role == agent:
    Join sites with agent_site_access
    Require:
        sites.id = site_id
        sites.tenant_id = user.tenant_id
        agent_site_access.user_id = user.id
    If found return allowed else denied
4. For any other role:
    return denied
```

۵.۴ فلوچارت

```
Start
|
```

```

v
Check Role
|
+-- super_admin -----> Allow
|
+-- customer_admin -----> Site belongs to tenant? -> Allow/Deny
|
+-- agent -----> Assigned in agent_site_access? -> Allow/Deny
|
+-- other -----> Deny

```

۶.۴ پیچیدگی

برای هر نقش فقط یک query محدود با 1 LIMIT اجرا می‌شود؛ بنابراین پیچیدگی عملی $O(1)$ با فرض وجود index روی `agent_site_access.user_id/site_id` و `sites.tenant_id, sites.id` است. فضای مصرفی $O(1)$ است.

۷.۴ مزایا

- منطق ساده و قابل audit.
- جداسازی روشن بین super، admin، customer admin و agent.
- مناسب برای SaaS چندمستاجری.

۸.۴ محدودیت‌ها

- اگر endpoint این helper را فراخوانی نکند، کنترل دسترسی از دست می‌رود؛ بنابراین audit endpointها مهم است.
- سطح دسترسی agent فقط بر اساس site است و permission جزئی‌تر مثل فقط مشاهده/فقط پاسخ در کد فعلی دیده نمی‌شود.

۵ الگوریتم ۳: شروع یا بازیابی مکالمه در Widget

۱.۵ هدف

این الگوریتم در endpoint `widget/conversation-start.php` یک مکالمه فعال برای `visitor` ایجاد یا بازیابی می‌کند. اگر `visitor` قبلاً مکالمه باز داشته باشد، همان مکالمه برگردانده می‌شود؛ در غیر این صورت محدودیت ماهانه پلن بررسی و مکالمه جدید ساخته می‌شود.

۲.۵ ورودی

- `site_key`
- `visitor_id`
- `source_page_title` و `source_page_url`
- `origin` درخواست widget

۳.۵ خروجی

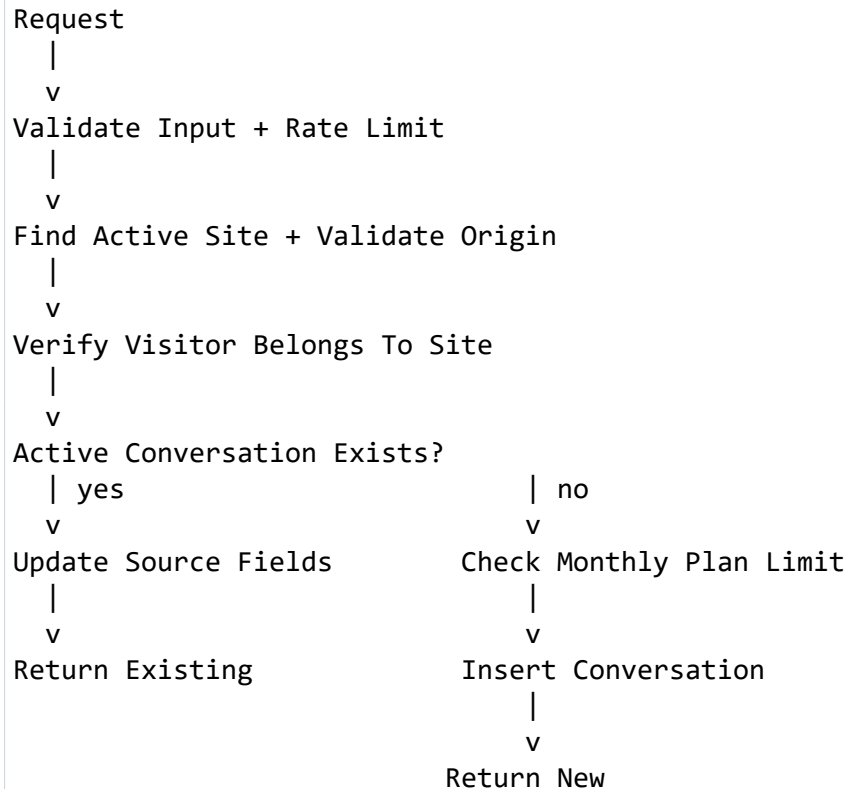
- شناسه مکالمه فعال، `site_id` و `visitor_id`.
- خطاهای `limit`، `plan` یا `found not site/visitor limit`، `rate validation`.

۴.۵ شبه‌کد

```
Algorithm START_OR_REUSE_WIDGET_CONVERSATION
Input: site_key, visitor_id, source_page_url, source_page_title
Output: conversation object

1. Require POST method
2. Validate site_key format and visitor_id
3. Validate source page fields length and URL scheme
4. Apply rate limit for site_key + visitor_id + remote IP
5. Load active site by site_key and active tenant
6. Validate widget origin against site domain
7. Verify visitor belongs to site
8. Search latest active conversation for site + visitor
9. If active conversation exists:
    update source page fields if provided
    return existing conversation
10. Else:
    ensure monthly conversation limit for site tenant
    insert new conversation with status = new
    return new conversation
```

۵.۵ فلوجارت



۶.۵ پیچیدگی

با index مناسب روی `visitor_id`، `site_key` و ستون‌های مکالمه شامل `visitor_id`، `site_id` و `status`، پیچیدگی هر درخواست $O(1)$ است. شمارش ماهانه در الگوریتم محدودیت پلن می‌تواند نسبت به تعداد مکالمات ماه جاری `tenant` خطی باشد، اما با index زمانی و `site_id` بهینه می‌شود.

۷.۵ مزایا

- جلوگیری از ساخت مکالمه تکراری برای یک `visitor`.
- حفظ `context` مکالمه فعال.
- اعمال `validation origin` و `limit rate` در مسیر عمومی.
- اتصال مستقیم به مدل درآمدی از طریق `limit. conversation monthly`.

۸.۵ محدودیت‌ها

- مکالمه فعال با وضعیت‌ها تعریف می‌شود؛ اگر وضعیت‌ها در آینده تغییر کنند باید این لیست به‌روزرسانی شود.
- در ترافیک خیلی بالا، ساخت همزمان مکالمه برای یک `visitor` می‌تواند نیازمند `constraint transaction` دقیق‌تر باشد.

۶ الگوریتم ۴: اعمال محدودیت‌های پلن

۱.۶ هدف

هدف این الگوریتم اعمال محدودیت‌های تجاری پلن روی قابلیت‌های محصول است: تعداد سایت‌ها، تعداد agentها، تعداد مکالمات ماهانه و قابلیت‌های AI.

۲.۶ ورودی

- tenant_id یا site_id.
- کلید قابلیت مانند ai_suggestions_enabled.
- جداول tenants، plans، sites، users و conversations.

۳.۶ خروجی

- ادامه عملیات در صورت مجاز بودن.
- پاسخ 403 یا 404 در صورت نبود پلن، غیرفعال بودن tenant/plan یا رسیدن به سقف.

۴.۶ شبه‌کد

```
Algorithm ENFORCE_PLAN_LIMIT
Input: tenant_id or site_id, limit type
Output: allowed or rejected

1. Load tenant joined with assigned plan
2. Reject if tenant not found or inactive
3. Reject if plan missing or inactive
4. For feature flag:
   if plan[feature_key] is false -> reject
5. For monthly conversations:
   count conversations for all tenant sites in current month
   if count >= max_monthly_conversations -> reject
6. For agents:
   count active users where role = agent
   if count >= max_agents -> reject
7. For sites:
   count sites for tenant
   if count >= max_sites -> reject
8. Otherwise allow
```

۵.۶ فلوجارت

```

Start
|
v
Load Tenant + Plan
|
+-- missing/inactive --> Reject
|
v
Select Limit Type
|
+-- Feature Flag --> Check Boolean --> Allow/Reject
+-- Monthly Conversations --> Count Current Month --> Allow/Reject
+-- Agents --> Count Active Agents --> Allow/Reject
+-- Sites --> Count Sites --> Allow/Reject

```

۶.۶ پیچیدگی

بار اصلی الگوریتم در `count` با `index` مناسب، پیچیدگی عملی نزدیک به $O(\log n)$ برای `lookup` و `index-based count` است. بدون `count index` می‌تواند $O(n)$ نسبت به تعداد کاربران، سایت‌ها یا مکالمات `tenant` باشد. فضای مصرفی $O(1)$ است.

۷.۶ مزایا

- اتصال مستقیم قابلیت‌های محصول به مدل درآمدی SaaS.
- قابل استفاده مجدد در های `endpoint` مختلف.
- کنترل قابلیت‌های AI و محدودیت‌های مصرفی از سمت `backend`.

۸.۶ محدودیت‌ها

- باید `audit` شود که همه های `endpoint` حساس واقعا های `helper` پلن را فراخوانی کنند.
- های `count` پرتکرار در مقیاس بالا بهتر است `cache` یا `aggregate` شوند.

۷ الگوریتم ۵: محدودسازی نرخ درخواست

۱.۷ هدف

این الگوریتم از spam force، brute و مصرف بیش از حد های endpoint عمومی یا حساس جلوگیری می‌کند. داده‌ها در جدول api_rate_limits ذخیره می‌شوند.

۲.۷ ورودی

- نام action مانند widget_ai_reply.
- identifier ساخته شده از agent user IP، و اطلاعات اضافی مثل site_key.
- maxAttempts و windowSeconds.

۳.۷ خروجی

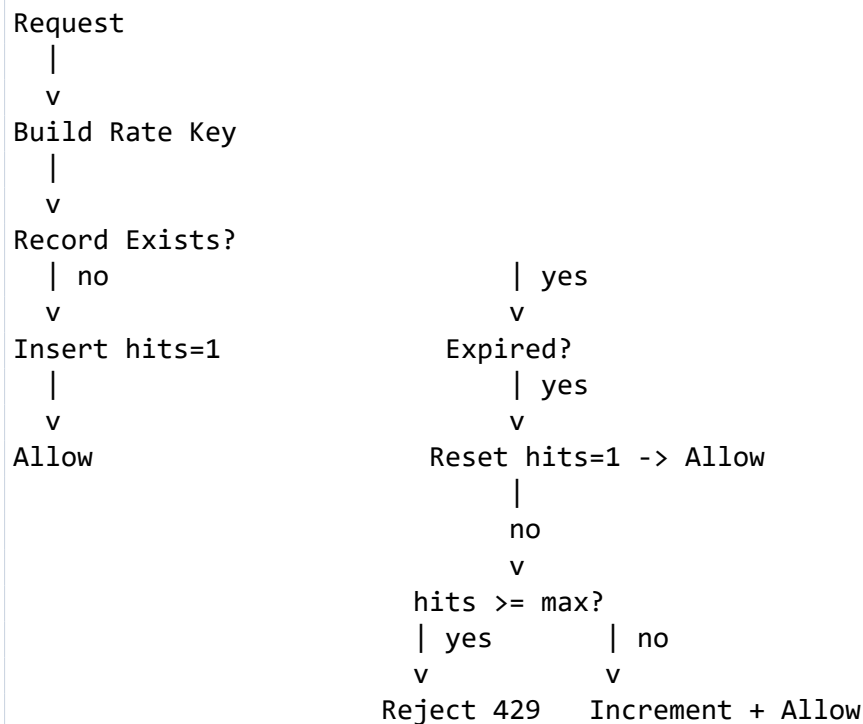
- اجازه ادامه درخواست.
- یا پاسخ 429 همراه با Retry-After.

۴.۷ شبه‌کد

```
Algorithm RATE_LIMIT
Input: action, identifier, max_attempts, window_seconds
Output: allowed or HTTP 429

1. If max_attempts <= 0 or window_seconds <= 0:
    allow
2. Build identifier_hash = SHA256(identifier)
3. Build rate_key = SHA256(action + identifier)
4. With 5% probability, delete up to 200 expired records
5. Load record by rate_key
6. If no record:
    insert record with hits = 1 and expires_at = now + window
    allow
7. If record expired:
    reset hits = 1 and expires_at = now + window
    allow
8. If hits >= max_attempts:
    return 429 with retry_after_seconds
9. Else:
    increment hits
    allow
```

۵.۷ فلوجارت



۶.۷ پیچیدگی

lookup و update با index روی rate_key عملاً $O(1)$ است. cleanup احتمالی حداکثر ۲۰۰ رکورد حذف می‌کند و با احتمال ۵ درصد اجرا می‌شود؛ بنابراین هزینه متوسط کنترل‌شده است. فضای مصرفی نسبت به تعداد action/identifier فعال $O(k)$ است.

۷.۷ مزایا

- ساده، قابل فهم و مناسب MVP.
- استفاده از hash برای identifier و جلوگیری از ذخیره مستقیم IP/user agent کامل.
- پشتیبانی از window و retry-after.

۸.۷ محدودیت‌ها

- window fixed است و نسبت به window sliding دقت کمتری در مرز پنجره دارد.
- در ترافیک بسیار بالا، update همزمان یک رکورد می‌تواند contention ایجاد کند.
- برای deployment distributed بهتر است Redis یا limiter rate متمرکز استفاده شود.

۸ الگوریتم ۶: خزیدن سایت و ساخت دانش

۱.۸ هدف

این الگوریتم صفحات سایت مشتری را از منابع تعریف‌شده دریافت می‌کند و دانش قابل جست‌وجو برای AI می‌سازد. خروجی آن جداول `ai_pages`، `ai_content_chunks`، `ai_terms` و `ai_generated_questions` است.

۲.۸ ورودی

- `site_id` و کاربر `admin.customer`
- منابع فعال در `ai_crawl_sources`: نوع `url`، `path_prefix` یا `sitemap`.
- تنظیمات `max_pages_per_crawl`، `max_depth` و `crawl_enabled`.

۳.۸ خروجی

- `crawl_run_id`
- تعداد، ها، `URLs` صفحات موفق، صفحات ناموفق، `terms chunks` و `ques-generated` tions.
- داده ذخیره‌شده در جداول AI.

۴.۸ شبه‌کد

```
Algorithm CRAWL_SITE_KNOWLEDGE
Input: site_id, active crawl sources, AI settings
Output: crawl summary and stored knowledge

1. Authenticate customer_admin and load site for tenant
2. Load AI settings; reject if crawl_enabled is false
3. Clamp max_pages to [1, 100] and max_depth to [0, 3]
4. Load active crawl sources
5. Insert ai_crawl_runs row with status = running
6. Build base URL from site domain
7. For each source:
    if type = url: add absolute URL to queue
    if type = path_prefix: add seed prefix URL to queue
    if type = sitemap: fetch sitemap and extract <loc> URLs
8. While queue is not empty and fetched_pages < max_pages:
    pop next URL
    skip if URL hash already visited
    skip if host does not belong to site domain
    fetch HTML
    if fetch failed: count failed and continue
    extract readable content and links
```

```

    if text length < 80: mark ignored and continue
    store page knowledge:
        upsert ai_pages
        delete old chunks for page
        split text into chunks
        extract terms for each chunk
        generate template questions
    if current depth < max_depth:
        normalize links and add same-domain links to queue
9. Update crawl run status = completed with counters
10. Update crawl sources last_crawled_at

```

۵.۸ فلوچارت

```

Start Crawl
|
v
Validate User + Site + Settings
|
v
Build URL Queue From Sources
|
v
Queue Loop
|
+-- visited/outside-domain --> Skip
|
+-- fetch failed -----> Count Failed
|
+-- short text -----> Ignore
|
v
Extract + Store Page Knowledge
|
v
Add Discovered Links If Depth Allows
|
v
Update Crawl Run Summary

```

۶.۸ پیچیدگی

اگر تعداد صفحات خزیده شده P ، تعداد لینک‌های بررسی شده L و طول کل متن‌ها T باشد، پیچیدگی زمانی کلی تقریباً $O(P + L + T)$ به علاوه هزینه شبکه است. قطعه‌بندی و استخراج term نسبت به طول متن خطی است. فضای مصرفی صف و مجموعه visited برابر $O(P + L_q)$ است که L_q لینک‌های داخل صف است، اما با $\text{max_pages_per_crawl}$ محدود می‌شود.

۷.۸ مزایا

- ساخت دانش از سایت مشتری بدون نیاز به ورود دستی همه .هاFAQ
- محدودسازی دامنه برای جلوگیری از crawl خارج از سایت مشتری.
- ثبت summary برای ارزیابی کیفیت .crawl
- تولید ،term chunk و سوال قالبی برای بهبود .retrieval

۸.۸ محدودیت‌ها

- crawl در درخواست HTTP اجرا می‌شود و برای سایت‌های بزرگ ممکن است timeout شود.
- verification SSL در fetch فعلی غیرفعال است و برای production باید بازبینی شود.
- جاوااسکریپت client-side صفحات render نمی‌شود؛ فقط HTML دریافت شده تحلیل می‌شود.

۹ الگوریتم ۷: نرمال‌سازی، قطعه‌بندی و استخراج واژه‌های مهم

۱.۹ هدف

این الگوریتم متن خام HTML را به واحدهای قابل جست‌وجو تبدیل می‌کند: متن نرمال، chunkهای محدود، های‌term پرتکرار و سوال‌های قالبی. این مرحله پایه کیفیت پاسخ‌دهی AI است.

۲.۹ ورودی

- HTML یا متن تمیز صفحه.
- metadata مانند title و heading.
- intent/category تشخیص‌داده‌شده.

۳.۹ خروجی

- متن نرمال‌شده.
- آرایه chunks با طول تقریبی ۸۰ تا ۶۵۰ کاراکتر.
- حداکثر ۴۰ term برتر برای هر chunk.
- سوال‌های template-based برای هر chunk.

۴.۹ شبه‌کد

```
Algorithm PREPARE_TEXT_KNOWLEDGE
Input: clean_text, title, heading, category, intent
Output: chunks, terms, generated_questions

1. Normalize text:
    decode HTML entities
    unify Arabic/Persian characters
    collapse repeated whitespace
2. Split text by lines
3. For each line:
    detect short heading-like lines
    accumulate body lines under current heading
    when body exceeds max length, emit chunk
4. If no chunks created:
    split by sentences and build chunks
5. For each chunk:
    lower-case and remove punctuation
    split into words
    remove stop words and short words
    count frequency
    score term = frequency * 10
    add boost if term appears in title or heading
```

```

    keep top 40 terms
6. Generate template questions based on intent
7. Store chunks, terms and questions

```

۵.۹ فلوجارت

```

Raw Text
|
v
Normalize
|
v
Line-Based Chunking
|
+-- no chunk --> Sentence-Based Fallback
|
v
Term Extraction + Scoring
|
v
Question Template Generation
|
v
Persist Knowledge Units

```

۶.۹ پیچیدگی

نرمال‌سازی، chunking و extraction term نسبت به طول متن $O(T)$ است. مرتب‌سازی $term$ ‌ها اگر تعداد $term$ ‌های متمایز U باشد $O(U \log U)$ است. چون خروجی $term$ ‌ها به ۴۰ محدود می‌شود، فضای نهایی کنترل شده است.

۷.۹ مزایا

- ساده، قابل توضیح و مناسب متن فارسی و انگلیسی.
- حفظ heading به عنوان زمینه chunk.
- تولید سوال‌های احتمالی برای افزایش شانس تطبیق سوال visitor.

۸.۹ محدودیت‌ها

- stemming، lemmatization و embedding وجود ندارد.
- تشخیص heading بر اساس heuristic ساده انجام می‌شود.
- سوال‌های تولیدی template-based هستند و تنوع زبانی محدود دارند.

۱۰ الگوریتم ۸: بازیابی و رتبه‌بندی پاسخ AI

۱.۱۰ هدف

هدف این الگوریتم یافتن بهترین پاسخ از میان دانش دستی، سوال‌های تولیدشده و chunks خزیده‌شده است. این الگوریتم در `ai_find_best_answer()` پیاده‌سازی شده و هسته AI فعال محصول محسوب می‌شود.

۲.۱۰ ورودی

- آجکت site شامل `id` و `tenant_id`.
- سوال `visitor` یا `agent`.
- جداول `knowledge_sources`، `ai_generated_questions`، `ai_content_chunks` و `ai_terms`.

۳.۱۰ خروجی

- `success`.
- `confidence_score`.
- پاسخ استخراجی.
- منابع برتر و های `candidate` برتر.
- شناسه، `question chunk` یا `source knowledge` تطبیق‌یافته.

۴.۱۰ شبه‌کد

```
Algorithm FIND_BEST_AI_ANSWER
Input: site, question
Output: answer result with confidence and sources

1. Normalize question
2. Extract non-stopword tokens, max 12 unique tokens
3. Detect question intent and category
4. If question is too short or tokens are empty:
    return no_answer
5. Search manual knowledge_sources by token LIKE conditions
6. Search ai_generated_questions by token LIKE conditions
7. Search ai_content_chunks by token LIKE conditions
8. Collect chunk IDs from candidates
9. Load term boosts for matched terms in candidate chunks
10. For each manual knowledge candidate:
    score = 0.45 * question_match
        + 0.35 * answer_match
        + 0.20 * all_text_match
```

```

    add manual knowledge boost
    add approved and FAQ boost
    add exact-ish question boost
11. For each generated question candidate:
    score = 0.60 * question_match + 0.25 * answer_match
    add intent/category boost
    add term boost
    add stored question score boost
12. For each content chunk candidate:
    score = 0.75 * content_match
    add intent/category boost
    add term boost
    add importance score boost
13. Clamp scores to 100
14. Sort candidates by score descending
15. If no candidate:
    return no_answer
16. Build extractive reply from best candidate answer_text
17. Return answer, confidence_score, top sources and best candidates

```

۵.۱۰ فلوچارت

```

Question
|
v
Normalize + Tokenize + Intent Detection
|
v
Search 3 Sources
|
+-- Manual Knowledge
+-- Generated Questions
+-- Content Chunks
|
v
Score + Boost Candidates
|
v
Sort Descending
|
+-- no candidate --> No Answer
|
v
Build Extractive Reply
|
v
Return Answer + Confidence + Sources

```

۶.۱۰ پیچیدگی

اگر تعداد توکن‌ها K ، های candidate دستی M ، سوال‌های تولیدی Q و C chunks باشد، scoring تقریباً $O(K(M+Q+C))$ است. هر query در کد به 50 LIMIT محدود شده، بنابراین در عمل $M, Q, C \leq 50$ و هزینه کنترل‌شده است. مرتب‌سازی روی حداکثر حدود 150 candidate انجام می‌شود: $O(N \log N)$.

۷.۱۰ مزایا

- قابل توضیح و قابل debug.
- اولویت دادن به دانش دستی تاییدشده.
- کاهش وابستگی به API خارجی.
- ثبت sources برای audit و بهبود کیفیت.

۸.۱۰ محدودیت‌ها

- جست‌وجو برداری و embedding ندارد؛ سوال‌های پارافرایز پیچیده ممکن است score پایین بگیرند.
- LIKE SQL برای متن‌های بزرگ در مقیاس بالا کافی نیست.
- پاسخ تولیدی extractive است و توان ترکیب چند منبع را به شکل LLM ندارد.

۱۱ الگوریتم ۹: پاسخ خودکار AI در Widget

۱.۱۱ هدف

این الگوریتم تصمیم می‌گیرد آیا AI اجازه دارد مستقیم به visitor پاسخ دهد یا باید fallback ارسال شود یا عملیات skip شود. این الگوریتم با احتیاط طراحی شده تا وقتی پشتیبان انسانی آنلاین است، AI مداخله مستقیم نکند.

۲.۱۱ ورودی

- message_id و conversation_id، visitor_id، site_key.
- آخرین پیام visitor.
- تنظیمات ai_site_settings.
- وضعیت آنلاین بودن admin customer یا agent.

۳.۱۱ خروجی

- پیام AI در جدول messages با sender_type = ai.
- یا skip با reason مشخص.
- لاگ در ai_answer_logs.
- ثبت unanswered در صورت score پایین.

۴.۱۱ شبه‌کد

```
Algorithm WIDGET_AI_AUTO_REPLY
Input: site_key, visitor_id, conversation_id, message_id
Output: AI message, fallback, or skipped result

1. Validate request method and input IDs
2. Validate site_key format
3. Apply rate limit for widget_ai_reply
4. Load conversation joined with active site and active tenant
5. Validate widget origin
6. Skip if conversation is closed
7. Skip if site.ai_mode == off
8. Load visitor message and verify sender_type = visitor
9. Skip if ai_answer_logs already has auto_reply/fallback for message_id
10. Load ai_site_settings
11. Skip if assistant_enabled is false
12. Skip if auto_reply_enabled is false
13. Count online support users in last 2 minutes
14. Skip if support is online
15. Call FIND_BEST_AI_ANSWER
```

```

16. If result success and score >= min_auto_reply_score:
    reply_mode = auto_reply
    reply_text = result.answer
17. Else:
    reply_mode = fallback
    reply_text = fallback_message
    insert ai_unanswered_questions
18. Insert AI message into messages
19. Insert ai_answer_logs
20. Update conversation status and last_message_at
21. Commit transaction and return result

```

۵.۱۱ فلوچارت

```

Widget AI Request
|
v
Validate + Rate Limit + Origin
|
v
Closed / AI Off / Assistant Disabled / Auto Disabled?
|
+-- yes --> Skip
|
v
Already Replied?
|
+-- yes --> Skip
|
v
Support Online?
|
+-- yes --> Skip
|
v
Find Best Answer
|
v
score >= threshold?
| yes | no
v     v
Insert AI Answer    Insert Fallback + Unanswered
|                  |
v                  v
Log + Update Conversation

```

۶.۱۱ پیچیدگی

پیچیدگی اصلی همان الگوریتم FIND_BEST_AI_ANSWER است. بررسی‌های قبل از آن‌های lookup محدود و $O(1)$ هستند. فضای مصرفی $O(N)$ برای AI candidate است که در کد محدود شده‌اند.

۷.۱۱ مزایا

- جلوگیری از پاسخ مستقیم AI هنگام آنلاین بودن پشتیبانی انسانی.
- جلوگیری از پاسخ تکراری برای یک پیام. visitor.
- fallback کنترل‌شده و ثبت سوال بی‌پاسخ برای بهبود دانش.
- تصمیم‌گیری مبتنی بر threshold. confidence

۸.۱۱ محدودیت‌ها

- آنلاین بودن پشتیبانی با window دو دقیقه‌ای محاسبه می‌شود؛ اگر presence دقیق نباشد تصمیم AI هم متأثر می‌شود.
- اگر fallback ارسال شود، visitor ممکن است پاسخ عمومی بگیرد نه پاسخ دقیق.
- برای جلوگیری کامل از condition race پاسخ تکراری، constraint دیتابیس روی mes-sage/mode می‌تواند مفید باشد.

۱۲ الگوریتم ۱۰: تولید پیشنهاد پاسخ برای Agent

۱.۱۲ هدف

این الگوریتم برای agent یا admin customer یک پاسخ پیشنهادی تولید می‌کند، اما پاسخ را مستقیم برای visitor ارسال نمی‌کند. این طراحی human-in-the-loop است و با کنترل، role plan و دسترسی site اجرا می‌شود.

۲.۱۲ ورودی

- conversation_id
- کاربر احراز شده با نقش customer_admin یا agent
- آخرین پیام visitor در مکالمه
- تنظیمات min_suggestion_score

۳.۱۲ خروجی

- رکورد در pending در ai_suggestions.
- confidence عددی بین ۰.۵ و ۱.۰.
- metadata، debug و sources
- لاگ در ai_answer_logs.

۴.۱۲ شبه‌کد

```
Algorithm GENERATE_AGENT_AI_SUGGESTION
Input: conversation_id, authenticated user
Output: pending AI suggestion

1. Require POST, authenticated user and allowed role
2. Validate conversation_id
3. Load conversation joined with site
4. Require plan feature ai_suggestions_enabled
5. Require user access to site
6. Reject if conversation is closed
7. Reject if site.ai_mode == off
8. Load latest visitor message for conversation
9. Load AI settings and min_suggestion_score
10. Call FIND_BEST_AI_ANSWER
11. If score >= min_suggestion_score:
    suggested_reply = result.answer
    reply_mode = suggestion
12. Else:
    suggested_reply = fallback_message
```

```

        reply_mode = no_answer
        insert ai_unanswered_questions if not duplicate
13. Convert confidence_score to 0.05..1.00 range
14. If pending suggestion exists for message:
    update it
15. Else:
    insert new pending ai_suggestions row
16. Insert ai_answer_logs
17. Return suggestion payload

```

۵.۱۲ فلوچارت

```

Agent Request
|
v
Auth + Role + Site Access + Plan Feature
|
v
Load Conversation + Latest Visitor Message
|
v
Find Best Answer
|
v
score >= min_suggestion_score?
| yes | no
|     |
v     v
Use Answer      Use Fallback + Save Unanswered
|               |
v               v
Upsert Pending Suggestion
|
v
Log and Return

```

۶.۱۲ پیچیدگی

پیچیدگی اصلی متعلق به retrieval/ranking است. سایر عملیات شامل چند lookup و upsert محدود است. در عمل با candidate های limit موجود، زمان پاسخ برای مکالمه معمولی کنترل شده است.

۷.۱۲ مزایا

- کاهش زمان پاسخ agent بدون حذف کنترل انسانی.
- ثبت accepted/edited/rejected در جریان‌های بعدی برای تحلیل کیفیت.
- استفاده از همان موتور پاسخ‌دهی مشترک با reply. auto

- جلوگیری از دسترسی غیرمجاز با feature. plan و access site role

۸.۱۲ محدودیت‌ها

- فقط آخرین پیام visitor را مبنای تولید پیشنهاد قرار می‌دهد.
- اگر base knowledge ضعیف باشد، خروجی fallback خواهد بود.
- کیفیت پیشنهاد به rule-based score وابسته است.

۱۳ الگوریتم ۱۱: اعتبارسنجی و ذخیره امن فایل پیوست

۱.۱۳ هدف

این الگوریتم فایل‌های ارسالی visitor یا agent را قبل از ذخیره بررسی می‌کند تا فایل‌های خطرناک، اجرایی، PDF HTML/SVG/script، دارای content active و تصویر نامعتبر وارد سیستم نشود.

۲.۱۳ ورودی

- آرایه فایل upload شده از \$_FILES.
- کانال ذخیره‌سازی مانند visitor، agent یا shared.
- تنظیمات UPLOAD_MAX_IMAGE_PIXELS و UPLOAD_MAX_BYTES.

۳.۱۳ خروجی

- metadata فایل ذخیره‌شده: نام اصلی، نام ذخیره‌شده، مسیر، MIME URL و اندازه.
- یا خطای validation با کد مناسب.

۴.۱۳ شبه‌کد

```
Algorithm SECURE_CHAT_ATTACHMENT_UPLOAD
Input: uploaded file, channel
Output: saved attachment metadata or rejection

1. Validate upload array shape and upload error code
2. Require tmp_name and is_uploaded_file
3. Read actual file size
4. Reject if size <= 0 or size > max allowed bytes
5. Sanitize original file name
6. Reject if any extension part is dangerous
7. Extract final extension and require it in allowed list
8. Scan file content prefix/full limited content:
    reject MZ executable
    reject PHP/script/svg/html/shebang payload
9. Detect MIME type with finfo
10. Require MIME type matches extension allowlist
11. If image:
    validate getimagesize
    validate image MIME
    reject oversized dimensions
12. If PDF:
    require %PDF- signature
    reject JavaScript, OpenAction, embedded files, launch actions, XFA
13. Normalize upload channel
14. Create upload directories by channel and date
```

15. Ensure .htaccess and index.html protection files
16. Generate random stored file name
17. Verify target directory stays under upload root
18. Move uploaded file and chmod 0644
19. Return public payload metadata

۵.۱۳ فلوجارت

```

Upload File
|
v
Validate Upload Status + Size + Name
|
v
Extension Allowlist + Dangerous Extension Check
|
v
Content Payload Scan
|
v
MIME Validation
|
+-- image --> Dimension Validation
|
+-- pdf ----> Signature + Active Content Validation
|
v
Create Protected Directory
|
v
Random File Name + Path Safety Check
|
v
Move File + Return Metadata

```

۶.۱۳ پیچیدگی

پیچیدگی زمانی نسبت به اندازه فایل $O(S)$ است، چون محتوای فایل تا سقف اندازه مجاز scan می‌شود. فضای مصرفی $O(S_{scan})$ است، اما S_{scan} با $UPLOAD_MAX_BYTES$ محدود می‌شود. بررسی MIME و تصویر هزینه ثابت یا وابسته به parser داخلی دارد.

۷.۱۳ مزایا

- چندلایه بودن اعتبارسنجی: extension، MIME، content و ابعاد.
- رد فایل‌های دوپسوندی مثل file.php.jpg.
- محافظت directory با .htaccess

- نام ذخیره‌شده random و غیرقابل حدس.

۸.۱۳ محدودیت‌ها

- جایگزین آنتی‌ویروس سازمانی نیست.
- scan کامل فایل برای حجم‌های بسیار بالا مناسب نیست؛ اما حجم upload فعلی محدود است.
- رفتار htaccess. فقط در وب‌سرورهای سازگار مثل Apache اثر کامل دارد.

۱۴ الگوریتم ۱۲: چرخه سوال بی‌پاسخ و بهبود دانش

۱.۱۴ هدف

این الگوریتم باعث می‌شود شکست AI به داده قابل اقدام تبدیل شود. وقتی score پاسخ پایین است، سوال در صف ai_unanswered_questions ذخیره می‌شود تا admin customer آن را بررسی و به base knowledge اضافه کند.

۲.۱۴ ورودی

- سوال. visitor.
- نتیجه FIND_BEST_AI_ANSWER.
- tenant_id, message_id, conversation_id و site_id.
- تصمیم admin برای reviewed, ignored یا added to knowledge.

۳.۱۴ خروجی

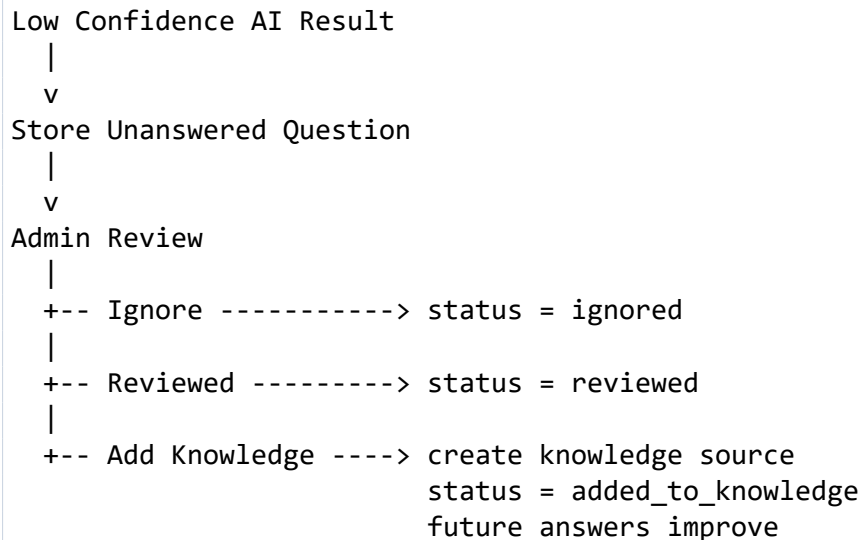
- رکورد unanswered با score و sources.
- رکورد جدید knowledge در صورت اضافه شدن به پایگاه دانش.
- بهبود پاسخ‌دهی در پرسش‌های آینده.

۴.۱۴ شبه‌کد

```
Algorithm UNANSWERED_IMPROVEMENT_LOOP
Input: low confidence question and admin decision
Output: improved knowledge base

1. During AI answer attempt:
    if score < threshold:
        insert ai_unanswered_questions
        include normalized question, detected intent/category,
        best score and best sources
2. Customer admin opens unanswered list
3. Admin reviews question and source context
4. If question should be ignored:
    update status = ignored
5. If question was handled externally:
    update status = reviewed
6. If question should improve AI:
    create knowledge source from question and answer
    update unanswered status = added_to_knowledge
7. Future AI searches include this manual knowledge
8. Manual knowledge receives scoring boost and can beat raw chunks
```

۵.۱۴ فلوجارت



۶.۱۴ پیچیدگی

ثبت سوال بی‌پاسخ $O(1)$ است. نمایش لیست unanswered به تعداد رکوردهای سایت و فیلتر status وابسته است. اضافه‌کردن knowledge نیز $O(1)$ است. اثر این الگوریتم در آینده با retrieval سنجیده می‌شود.

۷.۱۴ مزایا

- تبدیل خطای AI به فرایند بهبود مستمر.
- افزایش کیفیت دانش دستی که در رتبه‌بندی boost دارد.
- مناسب برای demo چون مسیر یادگیری سیستم قابل نمایش است.

۸.۱۴ محدودیت‌ها

- به اقدام admin customer وابسته است.
- اگر سوال‌ها بررسی نشوند، صف unanswered فقط رشد می‌کند.
- اضافه‌کردن دانش اشتباه می‌تواند کیفیت پاسخ‌های آینده را کاهش دهد.

۱۵ جمع‌بندی فنی

الگوریتم‌های پروژه AI Chat SaaS روی چند اصل طراحی شده‌اند: جداسازی tenant و site کنترل نقش و دسترسی، کاهش ریسک AI با threshold و fallback استفاده از دانش تاییدشده مشتری، ثبت لاگ و قابلیت بهبود تدریجی. موتور AI فعلی به صورت rule-based و extractive کار می‌کند و از الگوریتم‌های قابل توضیح برای جست‌وجو، امتیازدهی و رتبه‌بندی استفاده می‌کند.

برای ارزیابی فنی، مهم‌ترین نقاط قابل تاکید عبارت‌اند از:

- الگوریتم اختصاصی retrieval و scoring برای پاسخ‌گویی از دانش مشتری.
- چرخه کامل، crawl، chunk، question generated term، answer و ranking.
- کنترل human-in-the-loop از طریق پیشنهاد پاسخ agent.
- fallback و queue unanswered برای جلوگیری از پاسخ‌سازی بی‌پایه.
- الگوریتم‌های امنیتی عملی شامل JWT، site revoke، limit rate access و up-secure load.